ENGINEERING HANDOVER

Team Handover — One Transaction, Two Front Doors

How to write web, mobile and chatbot code so that every feature shares one schema, one validation, one API client and one backend service — with no duplicated logic.

React (Vite) · React Native (Expo) · Express BFF
CopilotKit + AG-UI · Zod · react-hook-form

Repository: unified-copilot-starter · Audience: web, mobile and backend engineers

Contents

1. What this repository is 3

2. Golden rules 5

3. The contract between the three surfaces 6

4. Add a new transaction (step-by-step) 7

5. The shared copilot hook in detail 11

6. Writing the two UI adapters 13

7. BFF conventions 14

8. Testing 15

9. Security and privacy 16

10. Troubleshooting 17

11. Definition of done (PR checklist) 18

12. Where to take this next 19

13. Commands 20

Web page · Mobile app · Embedded chatbot — one schema, one validation, one API, one business logic.

This document is the working guide for any engineer adding or changing a feature in this repository. Read section 2 (Golden rules) before writing code; read section 4 (Add a new transaction) when you start your first ticket.

1. What this repository is

We have one backend (BFF) that owns the business rules for a transaction, and three clients that all reach it the same way:



The important property: **the chatbot does not have its own implementation of anything.** It calls the same form store the user types into, and the same submit handler the button calls, which calls the same REST endpoint the page calls, which runs the same service function.

Repository layout

Path	Contents	Runs where
`packages/shared/src/schemas/`	Zod schemas — the single source of truth	everywhere
`packages/shared/src/fields/`	Field registry: label, step, on-screen location, visibility	everywhere
`packages/shared/src/api/`	API client + error codes + error store	everywhere
`packages/shared/src/form/`	`useOrderForm()` — the one form store	web + mobile
`packages/shared/src/copilot/`	`useOrderCopilot()` — all contexts and tools	web + mobile
`packages/bff/src/domain/`	Business rules (stock, pincode, coupons, totals)	server only
`packages/bff/src/routes/`	REST endpoints	server only

`packages/bff/src/copilot/`	CopilotKit runtime, system prompt, auth hook	server only
`apps/web/src/`	React page, driver.js adapter, confirm card	browser
`apps/mobile/src/`	RN screen, native adapter, spotlight, chat sheet	device
`tests/`	Shared-logic tests + chatbot tool tests	CI

2. Golden rules

1. **Never write a business rule twice.** Validation lives in the Zod schema; rules that need server data live in `domain/*Service.ts`. Clients never re-implement either.
2. **The chatbot writes to the form store, never to the UI.** `form.setValue()` on the same `react-hook-form` instance the inputs are bound to. No direct DOM/state poking.
3. **The chatbot submits by calling the page's submit handler.** One submit path, so loading state, error mapping and analytics stay identical.
4. **One auth, one token.** The chatbot endpoint uses the same bearer token as REST. The chatbot can never do more than the signed-in user.
5. **Anything the chatbot must know goes through `useAgentContext`.** Rendering data on screen does not put it in the model's context — those are two separate steps.
6. **The model is never trusted.** The BFF re-validates with the same schema and applies its own rules. A bad tool call gets the same error a bad form post gets.
7. **Never submit without an explicit human confirmation** (human-in-the-loop tool).
8. **Mask sensitive values** before they enter the context payload.

3. The contract between the three surfaces

Every transaction in this codebase exposes exactly this surface:

Artifact	File	Consumed by
`XxxSchema` (zod)	`schemas/xxx.ts`	form resolver, tool params, BFF validation
`XxxPatchSchema`	`schemas/xxx.ts`	chatbot `fillXxxForm` parameters
`XXX_FIELDS` registry	`fields/xxxFields.ts`	context (field map), `showField`, tests
`createApiClient().placeXxx()`	`api/client.ts`	page, mobile screen, chatbot tools
`useXxxForm()`	`form/useXxxForm.ts`	web page + mobile screen
`useXxxCopilot()`	`copilot/useXxxCopilot.ts`	web page + mobile screen
`placeXxx()` service	`bff/domain/xxxService.ts`	REST route (only)
REST routes	`bff/routes/xxx.ts`	clients
System prompt rules	`bff/copilot/runtime.ts`	the model

If a new feature needs a behaviour on the page, the app **and** the chatbot, it is added to this list — not to three separate places.

4. Add a new transaction (step-by-step)

Work in this order; each step is independently testable.

Step 1 — Write the schema

`packages/shared/src/schemas/<name>.ts`

```
import { z } from "zod";

export const RefundSchema = z.object({
  orderId: z.string().regex(/^ORD-[A-Z0-9]+$/, "Looks like ORD-XXXX").describe("Order id,
e.g. ORD-9F3K"),
  amount: z.coerce.number().positive().max(100000).describe("Refund amount in rupees"),
  reason: z.enum(["damaged", "wrong_item", "late", "other"]).describe("Why the customer wants
a refund"),
  notes: z.string().trim().max(300).optional().describe("Free-text notes"),
}).superRefine((v, ctx) => {
  if (v.reason === "other" && !v.notes) ctx.addIssue({ code: "custom", path: ["notes"],
message: "Describe the reason when choosing 'other'" });
});

export type RefundInput = z.input<typeof RefundSchema>;
export type Refund = z.output<typeof RefundSchema>;
export type RefundField = keyof RefundInput;

/** Subset the chatbot may fill. */
export const RefundPatchSchema = RefundSchema.partial().describe("Any subset of refund fields
collected from the user");
```

Rules for schemas:

- `.describe()` **every field.** That text is what the model reads when it decides what to ask for. Write it for a human, not for a compiler.
- Put the **user-facing message as the second argument** of every rule — it is shown in the form *and* returned to the chatbot so it can explain the problem.
- Use `z.coerce.number()` for numeric fields: chat text arrives as a string.
- Cross-field rules go in `.superRefine()`.
- Never `z.string()` for an enum the UI shows as a dropdown — use `z.enum`.

Step 2 — Register the fields

`packages/shared/src/fields/refundFields.ts`

```
import type { RefundField } from "../schemas/refund";
import type { FieldRegistry, StepMeta } from "./types";

export const REFUND_STEPS: StepMeta[] = [
  { step: 1, title: "Order" },
  { step: 2, title: "Refund details" },
```

```
];

export const REFUND_FIELDS: FieldRegistry<RefundField> = {
  orderId: { label: "Order id", step: 1, section: "Order lookup card, top field" },
  amount:  { label: "Refund amount", step: 2, section: "Refund block, first field" },
  reason:  { label: "Reason", step: 2, section: "Refund block, dropdown" },
  notes:   { label: "Notes", step: 2, section: "Refund block, last field", visibleWhen:
"Reason is 'other'" },
};
```

The registry is the model's **map of the screen**. It powers `showField` ("where do I enter this?") and lets the bot answer "what's on this page" without seeing any DOM.

Step 3 — Extend the API client

`packages/shared/src/api/client.ts`

```
return {
  // ...existing
  requestRefund: (r: RefundInput) => request<{ refundId: string; status: string }>("POST",
"/api/refunds", r),
  getOrder:     (orderId: string) => request<{ orderId: string; total: number; status: string
}>("GET", `/api/orders/${orderId}`),
};
```

Every failure is recorded automatically in `pageErrors` (source, endpoint, status, code, message, fieldErrors, details). **That store is what the chatbot reads to explain errors** — you get error explanations for free by using this client. Never call `fetch` directly from a screen or from a tool.

Add new codes to `api/errors.ts`:

```
export const ERROR_CODES = {
  // ...
  REFUND_WINDOW_CLOSED: "Refunds are only possible within 30 days of delivery.",
} as const;
```

Return error **codes**, not prose, from the BFF. The model turns the code into a sentence.

Step 4 — Implement the business rule once

`packages/bff/src/domain/refundService.ts`

```
import { RefundSchema, type RefundInput } from "@acme/shared";
import { DomainError } from "./orderService.js";

export function requestRefund(input: RefundInput, userId: string) {
  const parsed = RefundSchema.safeParse(input);           // same schema as the clients
  if (!parsed.success) {
    const fieldErrors = Object.fromEntries(parsed.error.issues.map((i) => [String(i.path[0]),
i.message]));
    throw new DomainError(422, { code: "VALIDATION_FAILED", message: "Some fields are
```

```
}
  const r = parsed.data;
  const order = loadOrder(r.orderId);                       // your data access
  if (!order || order.userId !== userId) throw new DomainError(404, { code:
"VALIDATION_FAILED", message: "Order not found", fieldErrors: { orderId: "Not found" } });
  if (daysSince(order.deliveredAt) > 30) throw new DomainError(409, { code:
"REFUND_WINDOW_CLOSED", message: "The 30-day refund window has closed." });
  if (r.amount > order.total) throw new DomainError(422, { code: "VALIDATION_FAILED",
message: "Refund cannot exceed the order total.", fieldErrors: { amount: `Max ₹
${order.total}` } });
  return { refundId: `RFD-${Date.now().toString(36).toUpperCase()}`, status: "requested" };
}
```

Then expose it: add the route in `bff/src/routes/` and mount it in `server.ts`. The chatbot gets the endpoint **for free** — it never needs a server-side tool.

Step 5 — Write the copilot hook

Copy `packages/shared/src/copilot/useOrderCopilot.ts` to `useRefundCopilot.ts` and change the four things that are transaction-specific:

```
export function useRefundCopilot({ hooks, form, ui, api, submit, ConfirmCard }: Args) {
  const { useAgentContext, useFrontendTool, useHumanInTheLoop } = hooks;
  const values = form.watch();

  // 1. CONTEXT — what the model can see
  useAgentContext({
    description: "Current refund screen state. Use it to answer questions; never invent field
values.",
    value: {
      screen: ui.screenName,
      currentStep: { number: ui.getStep(), title: REFUND_STEPS[ui.getStep() - 1]?.title },
      values: maskValues(values as Record<string, unknown>, REFUND_FIELDS),
      validationErrors: /* ... */,
      recentErrors: /* ... */,
    },
  });
  useAgentContext({ description: "Field map of the refund screen", value: { steps:
REFUND_STEPS, fields: REFUND_FIELDS, errorCodes: ERROR_CODES } });

  // 2. FILL — write into the same form store
  useFrontendTool({ name: "fillRefundForm", parameters: RefundPatchSchema, handler: /*
setValue + trigger + return invalid/missing */ });

  // 3. GUIDE — navigate + spotlight
  useFrontendTool({ name: "showField", parameters: z.object({ field: z.enum(fieldNames) }),
handler: /* ui.goToStep + ui.highlightField */ });

  // 4. LOOKUP — same BFF endpoints
  useFrontendTool({ name: "lookupOrder", parameters: z.object({ orderId: z.string() }),
handler: /* api.getOrder */ });

  // 5. SUBMIT — human confirms, then the page's own submit runs
  useHumanInTheLoop({ name: "confirmAndSubmitRefund", render: (p) =>
createElement(ConfirmCard, { ... }) });
}
```

Tool naming: `fill<Thing>Form`, `showField`, `goToStep`, `check<Thing>`, `confirmAndSubmit<Thing>`. Keep these names stable — the system prompt and your tests refer to them.

Always return a JSON string from a handler. The model reads fields by name; a bare "done" gives it nothing to work with.

Step 6 — Wire the web page

```
const form = useRefundForm();
const api = useMemo(() => createApiClient({ baseUrl: bffUrl, getToken }), [bffUrl,
getToken]);
const ui = useMemo(() => createWebUiAdapter({ getStep: () => stepRef.current, setStep }),
[]);

const submit = form.handleSubmit(async (values) => {
  try { setPlaced(await api.requestRefund(values)); }
  catch (e) {
    if (e instanceof ApiError && e.body.fieldErrors)
      for (const [f, msg] of Object.entries(e.body.fieldErrors)) form.setError(f as
RefundField, { message: msg });
    throw e;                        // re-throw: the confirm card reports it back to the bot
  }
});

useRefundCopilot({ hooks, form, ui, api, submit: () => submit(), ConfirmCard });
```

Two things make the binding work in the DOM: every field wrapper carries `data-field="<fieldName>"`, and the adapter looks fields up by that attribute. Add `data-field` to any new input — without it `showField` cannot find it.

Step 7 — Wire the mobile screen

The mobile screen is the same three lines (`useRefundForm`, `useRefundCopilot`, the same `api`), with two RN-specific differences:

- each field registers a `ref` into `fieldRefs` so the adapter can measure it;
- `createNativeUiAdapter(...)` is used instead of `createWebUiAdapter(...)`.

Everything else — schema, tools, prompt, endpoints — is shared.

Step 8 — Teach the system prompt

`packages/bff/src/copilot/runtime.ts` — add the new transaction's rules. Keep the existing structure: what the context means, which tool to call when, and the hard rule that the bot must never claim success the tool result did not confirm.

Step 9 — Test

Add cases to `tests/copilot.test.mts` following the existing pattern. See section 8.

---

5. The shared copilot hook in detail

5.1 Context: what the model sees every turn

Context entry	Purpose	Rules
Screen state	step, values (masked), validation errors, recent API/runtime errors	Keep it small. Never send the DOM.
Field map	label, step, section, help, `visibleWhen`, error codes	Static; lets the bot answer "where is X".

`useAgentContext` is **reactive**: pass the value from `form.watch()`, and every re-render publishes the current snapshot. Mount/unmount is automatic.

Two behaviours that cause silent bugs:

- The value is **stringified on the wire**. In an agent you must `JSON.parse` it.
- Rendering data on screen does **not** put it in context. If the bot should know it, register it explicitly.

5.2 The five tools

Tool	Type	Contract
`fillOrderForm`	frontend	Writes a subset into the form store, runs the same zod rules, returns `{applied, invalid, missing}`
`showField`	frontend	Navigates to the field's step and spotlights it; handles fields hidden by a condition
`goToStep`	frontend	Moves the wizard
`checkAvailability`	frontend	Read-only BFF lookups (stock, pincode)
`confirmAndSubmitOrder`	human-in-the-loop	Shows a summary card; on confirm runs the page's submit handler and reports the outcome

Writing a handler, the shape is always:

```
useFrontendTool({
  name: "fillOrderForm",
  description: "One sentence telling the model WHEN to call this, written for the model.",
  parameters: SomeSchema,                       // zod (Standard Schema) — same schema family as the form
  handler: async (args, { signal }) => {
    // 1. do the work through existing app code
    // 2. return a JSON string the model can reason about
    return JSON.stringify({ applied, invalid, missing });
```

```
  },
  }, [dependencies]);                              // re-register when deps change
```

Notes that matter:

- `deps` works like `useEffect`. Pass the form/api/ui objects so the tool always closes over the current ones.
- Use `available: false` to hide a tool in a state where it must not be called.
- Tools unmount with their component: put a step-specific tool inside that step's component and the model only sees it while the step is on screen.
- Read form errors with `form.getFieldState(name).error`, not `formState.errors`, inside a handler — the latter can be stale outside a render. (This bug was found and fixed in this repo; the test for it is in `tests/copilot.test.mts`.)

5.3 Human-in-the-loop submit

`useHumanInTheLoop` registers a tool with no handler; instead it renders a card and pauses the run until `respond(...)` is called. The pattern used here:

```
model: all fields valid → calls confirmAndSubmitOrder({ summary })
      ↓
card renders in the chat ("Confirm your order" + summary)
      ↓  user presses Place order
onConfirm → form.trigger()               ← the page's validation
         → submit()                      ← the page's submit handler
         → api.placeOrder()              ← the page's endpoint
respond({ submitted: true })             ← model learns the real outcome
```

On failure the card responds with `reason: "validation"` or `reason: "api"` plus the errors, and the model explains them. The bot can never claim an order was placed unless the handler said so.

---

6. Writing the two UI adapters

`UiAdapter` is the only platform-specific piece in the whole design:

```
export interface UiAdapter<F extends string = string> {
  screenName: string;
  getStep(): number;
  goToStep(step: number): void | Promise<void>;
  highlightField(field: F, title: string, description?: string): boolean | Promise<boolean>;
  focusField?(field: F): void;
}
```

Web (`apps/web/src/copilot/webUiAdapter.ts`) — `querySelector('[data-field="..."]')` , `scrollIntoView` , then `driver().highlight({ element, popover })` .

Mobile (`apps/mobile/src/copilot/nativeUiAdapter.ts`) — `view.measureLayout(scrollView)` , `scrollRef.scrollTo({ y })` , then render the `<Spotlight>` overlay inside the ScrollView content.

Adding a third surface (e.g. a desktop shell, a TV app) means writing one more adapter — nothing else in the shared layer changes.

7. BFF conventions

7.1 One service layer

```
routes/*.ts   → parse request, call service, map DomainError to HTTP
domain/*.ts   → all business rules; throws DomainError(status, {code, message, fieldErrors,
details})
auth.ts       → verifyToken() used by REST middleware AND the copilot hook
```

Never put a rule in a route handler. Never put HTTP concerns in a service.

7.2 The copilot endpoint

```
export const copilotHandler = createCopilotExpressHandler({
  runtime, basePath: "/api/copilotkit", cors: false,
  hooks: {
    onRequest: ({ request }) => {
      const user = verifyToken(request.headers.get("authorization")?.replace(/^Bearer /, "")
?? undefined);
      if (!user) throw new Response(JSON.stringify({ code: "UNAUTHORIZED", message: "Sign in
required" }), { status: 401, headers: { "Content-Type": "application/json" } });
    },
  },
});
```

Mount it **before** `express.json()` — the runtime reads the raw body stream itself.

The agent is created **per request** (`agents: () => ({ default: new BuiltInAgent(...) })`) because a single `BuiltInAgent` instance refuses concurrent runs. Keep that factory shape.

UI tools are **client-provided** — the server declares none. That is the whole trick: the same tool names exist on web and mobile, and the runtime only sees what the connected client advertises.

7.3 Prompt rules that keep the bot honest

The system prompt in `bff/src/copilot/runtime.ts` encodes these invariants — keep them when you edit it:

- The context is the truth; never claim a value that is not in it.
- Ask for anything reported as `missing`; explain anything reported as `invalid`.
- Use `showField` when the user cannot find something, and quote the field's location.
- For errors, read the code and `details` and offer the fix.
- Submit only through `confirmAndSubmitOrder`, and only report success if it returned `submitted: true`.
- Never ask the user to retype a masked value unless it is invalid.

8. Testing

Two suites, both plain `tsx` scripts — no test framework to configure.

```
npm run dev:bff      # terminal 1 (tests call the live BFF)
npm test             # terminal 2
```

8.1 Shared-logic tests (`tests/shared.test.mts`)

Cover the schema (accept/reject, coercions, cross-field rules), masking, and the API client including that a BFF error lands in `pageErrors` with the right code and details.

8.2 Chatbot tool tests (`tests/copilot.test.mts`)

Render the shared hook with **fake CopilotKit hooks** that capture the registrations:

```
const tools: Record<string, any> = {}; const contexts: Record<string, any> = {};
const hooks = {
  useFrontendTool: (t: any) => { tools[t.name] = t; },
  useHumanInTheLoop: (t: any) => { tools[t.name] = t; },
  useAgentContext: (c: any) => { contexts[c.description] = c.value; },
};
function Harness() {
  form = useOrderForm();
  const submit = form.handleSubmit(async (o) => { await api.placeOrder(o); submitted++; });
  useOrderCopilot({ hooks, form, ui, api, submit: () => submit(), ConfirmCard: () => null });
  return null;
}
await act(async () => { TestRenderer.create(createElement(Harness)); });
```

Then call `tools.fillOrderForm.handler({...})` directly and assert on the form store, the context payload, the adapter calls, and the submit outcome. This is how you prove "one logic" — the test exercises the same handler the model would call.

Every new tool gets at least one test covering: happy path, invalid input, and what the model receives back.

8.3 What the tests deliberately do not cover

A live LLM conversation (needs an API key), the app on a device, and the visual appearance of highlights. Those are manual checks — see section 11.

---

9. Security and privacy

Concern	Rule
Authorisation	Same bearer token for REST and the chatbot; verified in one place. Tools inherit the user's permissions — never add a tool that bypasses the API.
Trust	The model's arguments are untrusted input. The BFF re-validates with the same schema.
Sensitive data	Mark fields `sensitive: true` in the registry; `maskValues()` keeps them out of the context. Never send card numbers, passwords, OTPs or identity numbers.
Confirmation	Money-moving or destructive actions go through a human-in-the-loop tool.
Idempotency	Assume the model may retry. Give write endpoints an idempotency key before you ship.
Prompt injection	Text from users, pages or documents is data. The system prompt says so; do not let retrieved text override the rules.
Scope	Only register tools the user could perform themselves on the page.

10. Troubleshooting

Symptom	Cause	Fix
Chat replies but nothing happens on the page	Tool not registered, or `data-field` missing	Check the tool name in the network log; add `data-field` to the input
Bot says "filled" but a value is wrong	Handler returned no `invalid` / `missing` detail	Return JSON with per-field errors (use `form.getFieldState`)
`401` on `/api/copilotkit/info`	Token missing or not `Bearer`-prefixed	Check the provider `headers`; confirm `verifyToken` accepts it
Android app cannot reach the BFF	`localhost` on a device points at the device	Use `10.0.2.2` (emulator) or the LAN IP; HTTPS in production
`Unable to resolve module node:crypto` (RN release)	`jose` resolved to its Node build	Keep the `jose` → `browser` condition in `metro.config.js`
`Property 'ReadableStream' doesn't exist`	Polyfills loaded after CopilotKit	`react-native-get-random-values` then `/polyfills` on the first lines of the entry file
Streaming stalls in the app	Hermes lacks streaming `fetch`	Ensure the polyfill barrel is imported before any CopilotKit import
Web production build runs out of memory	Very large dependency graph	Build with more heap (`NODE_OPTIONS=--max-old-space-size=4096`) or bundle with esbuild
Model ignores a tool	Weak `description`, or the tool is not in context	Rewrite the description as "when to call this"; re-check the tool list on `/info`

11. Definition of done (PR checklist)

- [ ] Schema updated, every field `.describe()`d, user-facing messages present.
- [ ] Field registry updated (label, step, section, visibility).
- [ ] Business rule implemented **once**, in `domain/`.
- [ ] New error codes added to `ERROR_CODES`; BFF returns codes, not prose.
- [ ] Copilot hook registers context + tools; handler returns JSON with `invalid` / `missing`.
- [ ] Web page and mobile screen use the shared form + shared copilot hook (no duplicated logic).
- [ ] `data-field` / refs present on every new input.
- [ ] Submit still goes through the page's handler and the human-in-the-loop tool.
- [ ] System prompt updated for the new behaviour.
- [ ] Tests added for schema, API error capture and every new tool; `npm test` green.
- [ ] `npm run typecheck` green for shared, bff, web, mobile.
- [ ] Manually tried: fill by chat, ask "where is X", trigger a backend error and ask why.

12. Where to take this next

1. **Backend tools over MCP.** Today the tools live in the client. When you want the same operations available outside the page (Slack, an internal agent), expose the REST routes as MCP tools — `fastapi_mcp` (Python), Spring AI MCP (Java) or the MCP SDK (Node) — so the same endpoints serve both front doors. No business logic moves.
2. **Browser agents (WebMCP).** The tool definitions are already WebMCP-shaped (clear names, JSON Schema inputs, descriptions). CopilotKit can register the same tool on `document.modelContext` so browser agents can drive the page directly when the standard lands.
3. **Server-side persistence for threads.** Add a real `AgentRunner` (database-backed) to keep conversations across sessions.
4. **Generate the field registry from the schema.** The registry is mechanical; a small code generator would remove a manual step.

13. Commands

```
npm install
cp packages/bff/.env.example packages/bff/.env     # add your model API key
npm run dev:bff                                    # http://localhost:4000
npm run dev:web                                    # http://localhost:5173
npm test                                           # needs the BFF running
npm run typecheck                                  # shared + bff + web
```

Demo data: `SKU-1001` (12 in stock) · `SKU-2002` (2) · `SKU-3003` (0) · serviceable pincodes start with 56/40/11/60 · coupon `WELCOME10` · token `demo-<userId>`.

Questions about this design go to the team that owns the BFF. Changes to the shared schema are a contract change — review them like an API change.